

Tuning parameters and re-measuring

The four parameters the exam names, what each actually controls, and the discipline of measuring before and after.

Tuning is what you do **after** you have measured, and after the cheaper fixes are exhausted. Reaching for a config first is the classic mistake.

The four named parameters

PARAMETER	CONTROLS	DEFAULT
<code>spark.sql.shuffle.partitions</code>	Partitions produced by a shuffle in Spark SQL	200
<code>spark.default.parallelism</code>	Default partitions for RDD operations only	Total cores
<code>spark.executor.memory</code>	Memory available to each executor	Node type
<code>spark.driver.memory</code>	Memory available to the driver	Node type
<code>spark.sql.autoBroadcastJoinThreshold</code>	Size below which a table is auto-broadcast	10 MB

ON THE EXAM

Second row is the trap: `spark.default.parallelism` is an RDD setting.

spark.sql.shuffle.partitions

- ◆ **How many partitions a shuffle produces.** Every join, groupBy, distinct and orderBy is affected.
- ◆ **Default 200**, which is a number chosen years ago and unrelated to your cluster.
- ◆ **Too few:** each partition is large, executors strain, and you risk spill or out-of-memory errors.
- ◆ **Too many:** thousands of tiny tasks, scheduling overhead dominates, and parallelism does not improve.

ON THE EXAM

It is also the default partition count used by repartition when you do not supply a number.

AQE **coalesces partitions downwards**. It will not increase beyond the configured value. Set it too low and adaptive execution cannot rescue you.

Choosing a starting value

- ◆ **A common heuristic is one to three times the total core count**, so parallelism roughly matches the cluster.
- ◆ **Or target a partition size**: total shuffle bytes divided by a sensible partition size, often a few hundred megabytes.
- ◆ **Err high and let AQE coalesce**, since it can reduce but not raise.
- ◆ **On Databricks, 'auto' is a valid value**, enabling auto-optimised shuffle.

ON THE EXAM

The default of 200 is wrong in both directions: far too many on a two-core cluster, far too few on a thousand-core one.

spark.default.parallelism

- ◆ It applies to RDD operations, not to DataFrames or Spark SQL.
- ◆ So setting it does nothing to a DataFrame job. That is the whole trap.
- ◆ It defaults to the total number of worker cores.
- ◆ Know it because the exam names it, and because the distinction from shuffle.partitions is the point.

ON THE EXAM

On Databricks, `sc.defaultParallelism` reflects worker cores and cannot be overridden. Almost every question mentioning it is testing the RDD distinction.

Executor memory

- ◆ **How much memory each executor has** for processing and caching.
- ◆ **Too little and Spark spills to disk**, which is slow but survivable, or fails with an out-of-memory error.
- ◆ **On Databricks it usually follows the node type**, so choosing a memory-optimised node is the practical lever.
- ◆ **More memory is not the first fix for spill**. Often the partitions are simply too large.

ON THE EXAM

Spill and OOM are usually a partitioning problem wearing a memory costume. Check partition sizes before adding memory.

Driver memory

- ◆ **The driver plans work, tracks tasks and receives results.** It does not process partitions.
- ◆ **collect and toPandas pull data to the driver**, and a large result will exhaust it.
- ◆ **Broadcast joins collect the small side on the driver first**, so an oversized broadcast is a driver failure.
- ◆ **A driver OOM has different causes from an executor OOM**, and reading the error correctly saves hours.

ON THE EXAM

If the error names the driver, stop looking at executors. The cause is almost always collect, toPandas, or a broadcast.

The broadcast threshold, revisited

- ◆ `spark.sql.autoBroadcastJoinThreshold` is the static, plan-time threshold. Default 10 MB.
- ◆ Raising it lets larger dimension tables be broadcast, avoiding a shuffle on the large side.
- ◆ Raise it too far and you risk a driver out-of-memory error, since the small side is collected there.
- ◆ Set it to -1 to disable automatic broadcasting entirely.

ON THE EXAM

Databricks has a separate AQE threshold, `spark.databricks.adaptive.autoBroadcastJoinThreshold`, defaulting to 30 MB.

Where to set a parameter

SCOPE	HOW	WHEN
Cluster	Spark config in the cluster or job definition	A setting that should apply to everything on it
Session	<code>spark.conf.set(...)</code> in the notebook or job	One workload that needs different settings
Query	A hint, such as BROADCAST	One join, without changing anything global

ON THE EXAM

Narrower is safer. A cluster-wide change to fix one job affects every other job on that cluster.

The re-measure discipline

01

Baseline

Record the current runtime and where the time goes.

02

One change

Change a single parameter. Not three.

03

Re-run

Same data, same cluster, ideally a fresh session.

04

Compare

Did it help, and by how much?

05

Keep or revert

Revert anything that did not help. Do not accumulate.

NAMED IN THE OBJECTIVE

The exam objective says 're-measure the performance'. This is what that phrase means.

Measuring honestly

- ◆ **The first run is not comparable.** Caching, warm clusters and file system caches all flatter a repeat.
- ◆ **Use a fresh session** and run more than once before recording a number.
- ◆ **Wall-clock time includes cluster start-up.** On a cold classic cluster that can dominate a short job.
- ◆ **Look at stage-level metrics**, not just total duration. The bottleneck is usually one stage.

ON THE EXAM

A change that appears to halve the runtime has often just benefited from a warm cache. Repeat before believing it.

Symptom to parameter

SYMPTOM	LIKELY LEVER
Most cores idle during a join or aggregation	Raise <code>spark.sql.shuffle.partitions</code>
Thousands of tiny tasks, high overhead	Lower it, or let AQE coalesce
Disk spill in the Spark UI	More partitions first, then more memory
Executor out of memory	Partition sizes, then node type
Driver out of memory	A collect, a toPandas, or an oversized broadcast
A shuffle where a broadcast would do	Raise <code>autoBroadcastJoinThreshold</code> , or hint

ON THE EXAM

Note that two symptoms point away from parameters entirely. Driver OOM is a code problem, not a config one.

What to try before tuning

- ◆ **Read less.** Filter earlier, select fewer columns, use partition pruning.
- ◆ **Check the compute type.** A job on all-purpose compute is paying a premium before any tuning.
- ◆ **Look for fan-out.** A join multiplying rows makes everything downstream slower.
- ◆ **Check for an accidental full recompute,** such as a materialised view without row tracking.

THE ORDERING

Every item here changes how much work exists. A parameter only changes how the same work is divided up.

Tiebreakers from this module

WHEN A QUESTION SAYS

Setting shuffle.partitions low and expecting AQE to raise it

A DataFrame job unchanged by spark.default.parallelism

Cores idle during the shuffle stage

The error names the driver

One job on a shared cluster needs a change

Three parameters changed at once

The job reads more data than it needs

REACH FOR

AQE only coalesces downwards

That parameter applies to RDDs only

Too few shuffle partitions

collect, toPandas, or a broadcast

Session scope, not cluster scope

You cannot attribute the result. Change one.

Fix that before touching any parameter

ON THE EXAM

Seven lines. The first two are the ones most likely to appear as a question in their own right.

A nightly aggregation runs on a cluster with 240 total cores. The Spark UI shows the input stage using around 600 partitions and running well. The stage after the groupBy shows exactly 200 tasks, and most executors are idle during it.

That stage accounts for most of the job's runtime.

What should the team change?

- ☐ A Increase `spark.sql.shuffle.partitions`, which is at its default of 200
- ☐ B Increase `spark.default.parallelism` to match the core count
- ☐ C Increase executor memory so each of the 200 tasks completes faster
- ☐ D Add more workers to the cluster

◆ PAUSE AND DECIDE

What should the team change?

- A

 Increase `spark.sql.shuffle.partitions`
Correct. Exactly 200 tasks after a `groupBy` is the default showing through. With 240 cores, 200 tasks leaves capacity unused on a stage that dominates runtime.
- B

 Increase `spark.default.parallelism`
That parameter governs RDD operations. A `DataFrame groupBy` takes its partition count from `spark.sql.shuffle.partitions`, so this changes nothing.
- C

 Increase executor memory
Executors are idle, not struggling. Memory is not the constraint when most of the cluster has no work to do.
- D

 Add more workers
More cores sitting idle. The stage would still produce 200 tasks, so the extra capacity goes unused too.

THE SIGNAL

'Exactly 200' is the tell. A round, unchanging number in the Spark UI usually means a default is showing through.

An engineer reads that adaptive query execution tunes shuffle partitions automatically. To let it work freely, she sets `spark.sql.shuffle.partitions` to 40 on a 500-core cluster, reasoning that AQE will adjust upwards as needed.

The job slows considerably. The Spark UI shows 40 very large tasks and significant disk spill.

What went wrong?

- ☐ A AQE was disabled on this cluster and needs enabling
- ☐ B AQE coalesces partitions downwards only; it cannot increase beyond the configured value
- ☐ C 40 is below the minimum permitted value, so Spark ignored it
- ☐ D AQE only adjusts partitions for joins, not aggregations

◆ PAUSE AND DECIDE

What went wrong?

A AQE was disabled

AQE is on by default from DBR 7.3 onwards. And even fully enabled, it could not have raised 40 to a useful number.

C 40 is below a minimum

Invented limit. Spark accepted the value, which is exactly why there are 40 tasks.

B AQE coalesces downwards only

Correct. The configured value is an upper bound AQE works down from. Setting it low removes the headroom AQE needs.

D AQE only adjusts joins

It coalesces shuffle partitions generally, and also handles skew and join strategy. The limitation is directional, not by operation.

THE SIGNAL

The advice is to set it generously and let AQE reduce. Setting it low to 'let AQE decide' inverts how the mechanism works.

A job that builds a gold table takes 50 minutes. Investigating, an engineer finds it reads a full year of the silver table on every run and then filters to the last 7 days after a join.

He asks whether he should raise `spark.sql.shuffle.partitions` and increase executor memory to speed it up.

What is the right advice?

- ☐ A Yes; both changes will reduce the runtime on a job of this size
- ☐ B Filter to the 7 days before the join first, then re-measure; tuning parameters redistributes work rather than reducing it
- ☐ C Raise `shuffle.partitions` only, since memory is not the bottleneck
- ☐ D Convert the gold table to a materialised view instead

◆ PAUSE AND DECIDE

What is the right advice?

- A

 Yes, make both changes
Two changes at once, so neither can be attributed. And both redistribute work that should not exist in the first place.
- B

 Filter first, then re-measure
Correct. Reducing the data read is a change in how much work exists. Measure again afterwards, because the tuning question may vanish.
- C

 Raise shuffle.partitions only
Better than A because it changes one thing. Still tunes the division of work that a filter would eliminate.
- D

 Use a materialised view
Addresses repetition across readers, which is not the problem. One scheduled job reading too much data stays slow behind a materialised view.

THE SIGNAL

Spark can push some filters down, but a filter placed after a join still means joining a year of data to produce seven days.

A job fails with an out-of-memory error. The stack trace names the driver. The job joins a large fact table to a dimension table that has recently grown, and ends by calling display on the result.

A colleague suggests increasing `spark.executor.memory` and adding more workers.

What should be investigated first?

- ☐ A Increase executor memory as suggested, since OOM means insufficient memory
- ☐ B The error names the driver, so look at the broadcast of the grown dimension table, and at the display call
- ☐ C Increase `spark.sql.shuffle.partitions` to make each partition smaller
- ☐ D The dimension table needs OPTIMIZE to reduce its file count

◆ PAUSE AND DECIDE

What should be investigated first?

- A

 Increase executor memory
The error names the driver. Adding executor memory addresses a component that is not failing, at permanent cost.
- B

 Look at the broadcast and the display call
Correct. Both pull data to the driver. A grown dimension table may still be broadcast, and display collects results back.
- C

 Raise shuffle.partitions
Shuffle partitions affect executor-side memory pressure. They do nothing for the driver, which does not process partitions.
- D

 Run OPTIMIZE on the dimension table
File layout affects read efficiency. It does not change how much the driver has to hold.

THE SIGNAL

Reading which component failed is the whole diagnosis. Driver and executor OOM have entirely different causes and fixes.

Measure, change one thing, measure again. And before any of that, ask whether the work itself can be made smaller.

Parameters divide work; they do not reduce it.